

LAPORAN PRAKTIKUM PEMOGRAMAN WEB

MODUL 6: Pemrograman Berorientasi Objek (OOP) pada PHP



Oleh:

Zalfaa Ghaitsaa Arrahma Bahtiar

707012500077

D4SIKC-49-04

**TELKOM UNIVERSITY
2026**

A. TUJUAN PRAKTIKUM

Setelah menyelesaikan modul ini, peserta diharapkan mampu:

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Menggunakan tipe data object dalam PHP.
3. Membuat dan menggunakan class serta object.
4. Memahami konsep constructor dan destructor.
5. Menerapkan konsep inheritance dalam OOP PHP.

B. ALAT DAN BAHAN

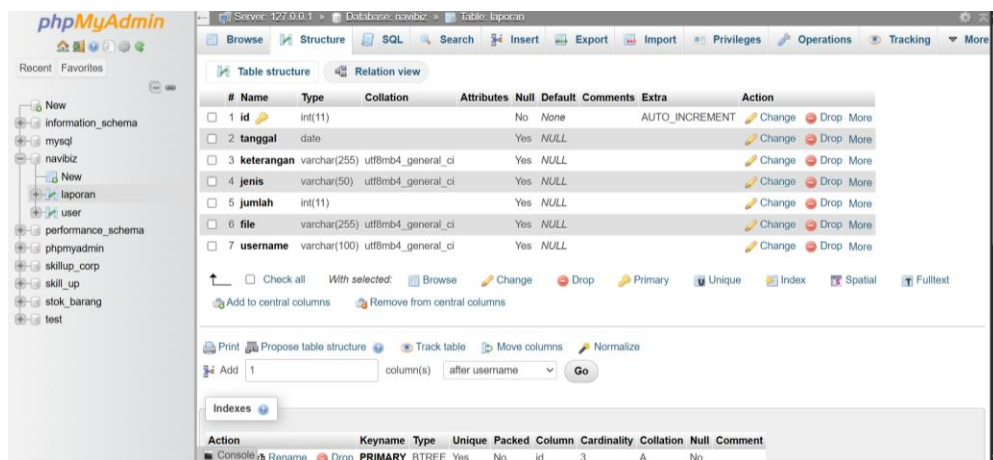
Untuk melaksanakan praktikum ini, diperlukan:

1. Komputer dengan sistem operasi Windows, macOS, atau Linux.
2. Web server seperti XAMPP.
3. Editor teks atau IDE seperti VS Code, Sublime Text, atau PHPStorm.
4. Browser web (Google Chrome, Mozilla Firefox, atau lainnya).

C. LANGKAH LANGKAH PRAKTIKUM

1) Studi Kasus 1

- a) Menambahkan data file dan username ke dalam database menggunakan phpMyAdmin
- b) Create database -> buat table user -> masukkan data dan tipenya, hingga tampilannya seperti ini



Gambar C.1.1 Tabel laporan pada database navibiz

- c) Lalu buat code nya di VS Code

```

1  <?php
2  session_start(); // memulai session untuk simpan data login
3  require "Database.php"; // memanggil class koneksi database OOP
4  require "Auth.php"; // memanggil class autentikasi user login
5
6  $db = new Database(); // membuat objek database untuk koneksi
7  $auth = new Auth($db->conn); // kirim koneksi ke class Auth
8
9  $username = ""; // inisialisasi variabel username kosong awal
10
11 // ambil dari cookie
12 if(isset($_COOKIE['username'])){ // cek apakah cookie username tersedia
13     $username = $_COOKIE['username']; // isi username dari cookie browser
14 }
15
16 // kalau sudah login
17 if(isset($_SESSION['username'])){ // cek apakah user sudah login sebelumnya
18     header("Location: dashboard.php"); // redirect ke halaman dashboard utama
19     exit; // hentikan eksekusi script setelah redirect
20 }
21
22 $error = ""; // variabel untuk menyimpan pesan error login
23
24 // proses login
25 if(isset($_POST['login'])){ // cek apakah tombol login ditekan user
26     $username = $_POST['username']; // ambil input username dari form
27     $password = $_POST['password']; // ambil input password dari form
28
29     if($auth->login($username, $password)){ // cek login ke database user
30
31         echo "<script>
32             alert('Login berhasil!'); // tampilkan notifikasi login berhasil
33             window.location='dashboard.php'; // arahkan ke dashboard setelah login
34             </script>";
35     } else {
36         $error = "Username atau password salah!"; // simpan pesan error login gagal
37     }
38 }
39 ?>

```

Gambar C.1.2 Code pada file login2.php

```

194 <?php
195 require "database.php"; // memanggil class koneksi database OOP
196 require "User.php"; // memanggil class user untuk registrasi
197
198 $db = new Database(); // membuat objek database untuk koneksi
199 $user = new User($db->conn); // kirim koneksi ke class User
200
201 $message = ""; // variabel untuk menyimpan pesan hasil proses
202
203 if(isset($_POST['submit'])){ // cek apakah form sudah disubmit user
204
205     // ambil data dari form
206     $data = [ // array untuk menampung semua input user
207         "name" => $_POST['name'], // ambil input nama dari form
208         "email" => $_POST['email'], // ambil input email dari form
209         "username" => $_POST['username'], // ambil input username dari form
210         "password" => $_POST['password'], // ambil input password dari form
211         "telepon" => $_POST['telepon'], // ambil input nomor telepon user
212         "alamat" => $_POST['alamat'] // ambil input alamat dari form
213     ];
214
215     // cek username
216     if($user->cekUsername($data['username'])){ // cek apakah username sudah terdaftar
217         $message = "Username sudah digunakan!"; // set pesan jika username sudah ada
218     } else {
219
220         if($user->register($data)){ // simpan data user ke database
221             $message = "Registrasi berhasil!"; // pesan jika berhasil registrasi
222         } else {
223             $message = "Terjadi error!"; // pesan jika gagal simpan data
224         }
225     }
226 }
227 ?>

```

Gambar C.1.3 Code pada file register.php

```

1 <?php
2 class Session { // class untuk mengatur session user login
3
4     public function checkLogin(){ // fungsi untuk cek apakah user sudah login
5         if(!isset($_SESSION['username'])){ // jika session username tidak tersedia
6             header("Location: login2.php"); // redirect ke halaman login
7             exit; // hentikan eksekusi setelah redirect
8         }
9     }
10
11     public function getUser(){ // fungsi untuk ambil username login
12         return $_SESSION['username']; // kembalikan nilai username dari session
13     }
14
15     public function checkCookie(){ // fungsi untuk cek login dari cookie
16         if(isset($_COOKIE['username']) && !isset($_SESSION['username'])){ // jika cookie ada tapi session belum
17             $_SESSION['username'] = $_COOKIE['username']; // set session dari cookie yang tersimpan
18         }
19     }
20 }
21 ?>

```

Gambar C.1.4 Code OOP Session.php

```

2  class User { // class untuk mengelola data user database
3      private $conn; // properti untuk menyimpan koneksi database aktif
4
5      public function __construct($db){ // constructor menerima koneksi dari luar class
6          $this->conn = $db; // simpan koneksi database ke properti class
7      }
8
9      // cek username sudah ada atau belum
10     public function cekUsername($username){ // fungsi untuk cek username di database
11         $result = mysqli_query($this->conn, // jalankan query ke database MySQL
12             "SELECT * FROM user WHERE username='$username'" // query cek username sudah ada
13         );
14         return mysqli_num_rows($result) > 0; // true jika username sudah ditemukan
15     }
16
17     // insert data user
18     public function register($data){ // fungsi untuk menyimpan data user baru
19         return mysqli_query($this->conn, // jalankan query insert ke database
20             "INSERT INTO user (name, email, username, password, telepon, alamat)
21             VALUES (
22                 '{$data['name']}',
23                 '{$data['email']}',
24                 '{$data['username']}',
25                 '{$data['password']}',
26                 '{$data['telepon']}',
27                 '{$data['alamat']}'
28             )"
29         );

```

Gambar C.1.5 Code OOP User.php

```

Pekan-06 > Auth.php
1  <?php
2  class Auth {
3      private $conn; // menyimpan koneksi database
4
5      public function __construct($db){
6          $this->conn = $db; // inisialisasi koneksi database ke class
7
8          // inisialisasi koneksi database ke class
9          if(session_status() == PHP_SESSION_NONE){
10             session_start(); // mulai session jika belum aktif
11         }
12     }
13
14     // LOGIN
15     public function login($username, $password, $remember = false){
16         // mengamankan input dari SQL injection
17         $username = mysqli_real_escape_string($this->conn, $username);
18         $password = mysqli_real_escape_string($this->conn, $password);
19         // query akan cek user di database
20         $query = mysqli_query($this->conn,
21             "SELECT * FROM user WHERE username='$username' AND password='$password'"
22         );
23
24         // jika data ditemukan (Login berhasil)
25         if(mysqli_num_rows($query) > 0){
26             $_SESSION['username'] = $username; //simpen username ke session
27
28             if($remember){ //jika user centang remember me
29                 setcookie("username", $username, time() + (86400 * 7), "/"); //simpan username ke cookie selama 7 hari
30             }

```

```

Pekan-06 > Auth.php
2  class Auth {
15  public function Login($username, $password, $remember = false){
25      if(mysqli_num_rows($query) > 0){
32          return true; //kembalikan true jika berhasil
33      }
34
35      return false; //kembalikan false jika gagal
36  }
37
38  // CEK LOGIN
39  public function isLogin(){
40      //cek jika username tersedia
41      return isset($_SESSION['username']);
42  }
43
44  // LOGOUT
45  public function Logout(){
46      session_destroy(); //hapus semua data session
47      //hapus cookie dengan waktu kadaluarsa
48      setcookie("username", "", time() - 3600, "/");
49      header("Location: login2.php"); //redirect ke halaman Login
50      exit;
51  }
52
53  // AMBIL USER LOGIN
54  public function user(){
55      //ambil username dari session jika ada
56      return $_SESSION['username'] ?? null;
57  }
58

```

Gambar C.1.6 Code Auth.php

```

1  <?php
2  require "Database.php"; // memanggil class database
3  require "Auth.php"; // memanggil class auth login logout
4
5  $db = new Database(); // membuat objek database
6  $auth = new Auth($db->conn); // kirim koneksi ke class Auth
7
8  $auth->Logout(); // jalankan fungsi logout user
9  ?>

```

Gambar C.1.7 Tampilan logout.php

Pekan-06 > 🐘 dashboard.php

```
1  <?php
2  session_start();
3  include "Session.php";
4
5  $session = new Session(); // untuk membuat object
6
7  $session->checkCookie(); // cek cookie dulu
8  $session->checkLogin();  // cek login
9
10 $username = $session->getUser(); // untuk mengambil username
11 ?>
```

Gambar C.1.8 Tampilan Dasboard.php

```
1  <?php
2  session_start();
3
4  require "database.php"; //memanggil file koneksi database
5  require "Laporan.php"; //memanggil class Laporan OOP
6
7  $db = new Database(); //membuat objek database baru
8  $laporan = new Laporan($db->conn); //kirim koneksi ke class laporan
9
10 // ambil session dari cookie jika belum ada
11 if(isset($_COOKIE['username']) && !isset($_SESSION['username'])){
12     $_SESSION['username'] = $_COOKIE['username']; //set session dari cookie
13 }
14
15 if(!isset($_SESSION['username'])){ //cek jika user belum Login
16     header("Location: login.php"); //redirect ke halaman login
17     exit;
18 }
19
20 $username = $_SESSION['username']; //ambil username dari session
21
22 // ===== DELETE =====
23 if(isset($_GET['hapus'])){ //cek apakah ada parameter hapus
24     $laporan->delete($_GET['hapus'], $username); //hapus data berdasarkan id
25
26     echo "<script>
27     alert('Data berhasil dihapus');
28     window.location='dompet.php';
29     </script>"; //notif dan reload halaman
```

```

23  if(isset($_GET['hapus'])){ //cek apakah ada parameter hapus
30  }
31
32  // ===== EDIT =====
33  $editData = null; //default data edit kosong
34
35  if(isset($_GET['edit'])){ //cek apakah mode edit aktif
36      $editData = $Laporan->getId($_GET['edit']); //ambil data berdasarkan id
37  }
38
39  // ===== INSERT / UPDATE =====
40  if(isset($_POST['tanggal'])){ //cek apakah form dikirim
41
42      if(isset($_POST['id']) && $_POST['id'] != ""){ //cek apakah update ada id
43          $Laporan->update(
44              $_POST['id'], //id data yang akan di upload
45              $_POST, //data form yang dikirim
46              $_FILES['file'], //file di upload dari form
47              $username, //username pemilik data
48              $editData['file'] ?? "" //file Lama jika tidak diganti
49          );
50
51          echo "<script>
52          alert('Data berhasil diupdate');
53          window.location='dompet.php';
54          </script>"; //notif update berhasil
55
56      } else {
57          $Laporan->insert($_POST, $_FILES['file'], $username); //proses insert baru
58
59          echo "<script>
60          alert('Data berhasil ditambahkan');
61          window.location='dompet.php';
62          </script>"; //notif insert berhasil
63      }
64  }
65
66  // ===== TAMPIL DATA =====
67  $dataLaporan = $Laporan->getAll($username); //ambil semua data milik user
68  ?>

```

Gambar C.1.9 Tampilan dompet.php


```

1  <?php
2  class Database { // class untuk mengelola koneksi database MySQL
3      private $host = "localhost"; // alamat server database biasanya localhost
4      private $user = "root"; // username default MySQL pada XAMPP
5      private $pass = ""; // password MySQL default kosong XAMPP
6      private $db = "navibiz"; // nama database yang akan digunakan
7
8      public $conn; // properti untuk menyimpan hasil koneksi database
9
10     public function __construct(){ // constructor otomatis saat objek dibuat
11         $this->conn = mysqli_connect( // fungsi untuk membuat koneksi database
12             $this->host, // parameter host server database
13             $this->user, // parameter username database
14             $this->pass, // parameter password database
15             $this->db // parameter nama database tujuan
16         );
17
18         if(!$this->conn){ // cek apakah koneksi database berhasil dibuat
19             die("Koneksi gagal: " . mysqli_connect_error()); // hentikan program jika koneksi gagal
20         }
21     }
22 }
23 ?>

```

Gambar C.1.10 Tampilan code Database.php yang terhubung dengan file dompet.php

```

1  <?php
2  class Laporan { // class untuk mengelola data laporan user
3      private $conn; // properti untuk menyimpan koneksi database aktif
4
5      public function __construct($db){ // constructor menerima koneksi dari luar
6          $this->conn = $db; // simpan koneksi ke dalam properti class
7      }
8
9      // ambil semua data
10     public function getAll($username){ // fungsi mengambil semua data berdasarkan user
11         return mysqli_query($this->conn, // jalankan query ke database MySQL
12             "SELECT * FROM laporan WHERE username='$username' ORDER BY id DESC" // ambil dataurut terbaru
13         );
14     }
15
16     // ambil 1 data
17     public function getById($id){ // fungsi mengambil satu data berdasarkan id
18         $result = mysqli_query($this->conn, // jalankan query select satu data
19             "SELECT * FROM laporan WHERE id=$id" // ambil data sesuai id tertentu
20         );
21         return mysqli_fetch_assoc($result); // ubah hasil query jadi array
22     }
23
24     // hapus data
25     public function delete($id, $username){ // fungsi untuk menghapus data laporan
26         return mysqli_query($this->conn, // jalankan query delete database
27             "DELETE FROM laporan WHERE id=$id AND username='$username'" // hapus data milik user tertentu
28         );
29     }
30
31     // upload file
32     public function uploadFile($file, $oldFile = ""){ // fungsi upload file ke folder server
33         if($file['name'] != ""){ // cek apakah file baru diupload user
34             $namaFile = $file['name']; // ambil nama file yang diupload
35             move_uploaded_file($file['tmp_name'], "uploads/" . $namaFile); // pindahkan file ke folder uploads
36             return $namaFile; // kembalikan nama file baru disimpan
37         }
38         return $oldFile; // jika tidak upload gunakan file lama
39     }
40
41     // insert data
42     public function insert($data, $file, $username){ // fungsi untuk menambah data baru laporan
43         $fileName = $this->uploadFile($file); // proses upload file terlebih dahulu
44
45         return mysqli_query($this->conn, // jalankan query insert ke database
46             "INSERT INTO laporan (tanggal,keterangan,jenis,jumlah,file,username)
47             VALUES (
48                 '{$data['tanggal']}',
49                 '{$data['keterangan']}',
50                 '{$data['jenis']}',
51                 '{$data['jumlah']}',
52                 '$fileName',
53                 '$username'
54             )");
55     }
56

```

```

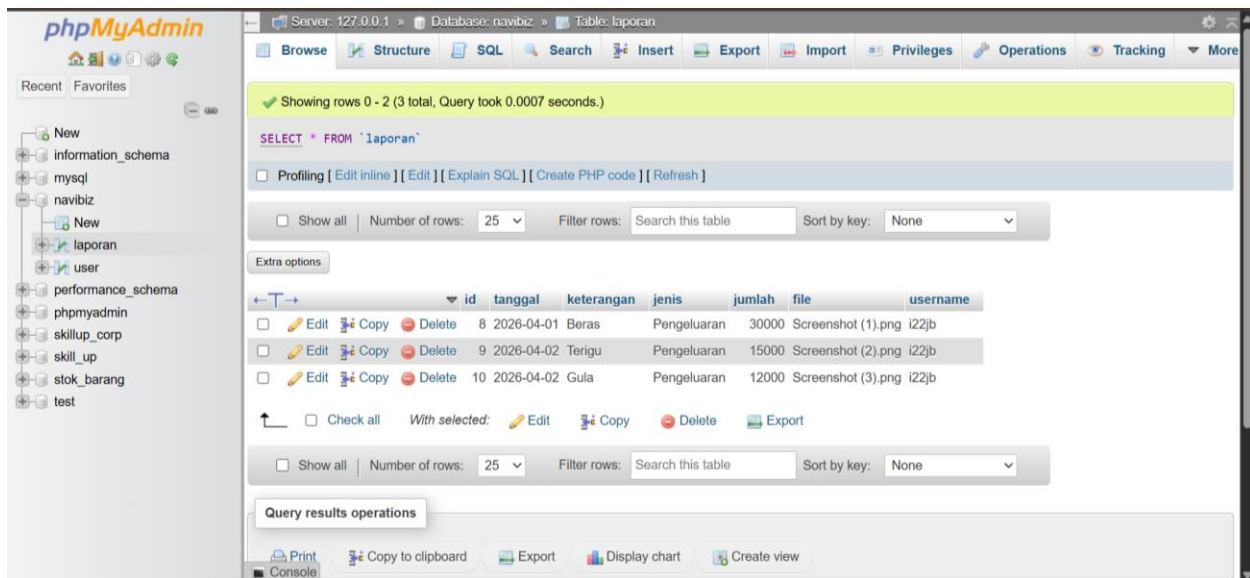
57 // update data
58 public function update($id, $data, $file, $username, $oldFile){ // fungsi update data Laporan berdasarkan id
59     $fileName = $this->uploadFile($file, $oldFile); // upload file baru atau gunakan lama
60
61     return mysqli_query($this->conn, // jalankan query update ke database
62         "UPDATE laporan SET
63             tanggal='{$data['tanggal']}',
64             keterangan='{$data['keterangan']}',
65             jenis='{$data['jenis']}',
66             jumlah='{$data['jumlah']}',
67             file='{$fileName}'
68             WHERE id=$id AND username='$username'"); // update hanya milik user tersebut
69     }
70 }
71 ?>

```

Gambar C.1.9 Tampilan code Laporan.php yang terhubung dengan file dompet.php

D. HASIL DAN DOKUMENTASI PROGRAM

1) Hasil Studi Kasus 1



The screenshot shows the phpMyAdmin interface for a database named 'navibiz'. The 'laporan' table is selected, and its structure and data are displayed. The table has 8 columns: id, tanggal, keterangan, jenis, jumlah, file, and username. The data shows three rows of transactions.

id	tanggal	keterangan	jenis	jumlah	file	username
8	2026-04-01	Beras	Pengeluaran	30000	Screenshot (1).png	i22jb
9	2026-04-02	Terigu	Pengeluaran	15000	Screenshot (2).png	i22jb
10	2026-04-02	Gula	Pengeluaran	12000	Screenshot (3).png	i22jb

Gambar D.1.1 Tabel database

2024-09-01

Pembelian Bahan

Pengeluaran

500000

Laporan manual

ut transaksi manual dan lihat rekapnya.

Input data

Tanggal

hh/bb/yyyy

Keterangan

Jenis

Pemasukan

Jumlah

Bukti Transaksi

Pilih File Tidak ada file yang dipilih

Simpan

Reset

Data laporan manual

Tanggal	Keterangan	Jenis	Jumlah	Bukti Transaksi	Aksi
2026-04-02	Gula	Pengeluaran	12000	Lihat	Edit Hapus
2026-04-02	Terigu	Pengeluaran	15000	Lihat	Edit Hapus
2026-04-01	Beras	Pengeluaran	30000	Lihat	Edit Hapus

Gambar D.1.2 Tampilan form di halaman dompet

2024-09-01

Pembelian Bahan

Pengeluaran

500000

Laporan manual

ut transaksi manual dan lihat rekapnya.

Input data

Tanggal

hh/bb/yyyy

Keterangan

Jenis

Pemasukan

Jumlah

Bukti Transaksi

Pilih File Tidak ada file yang dipilih

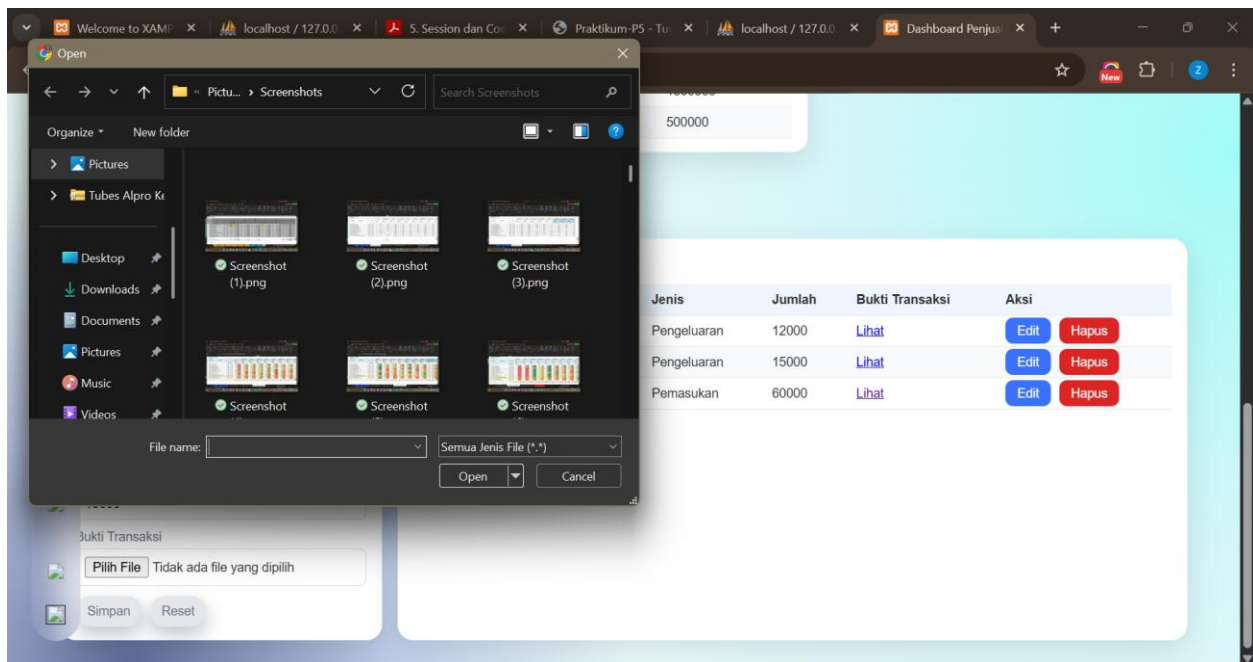
Simpan

Reset

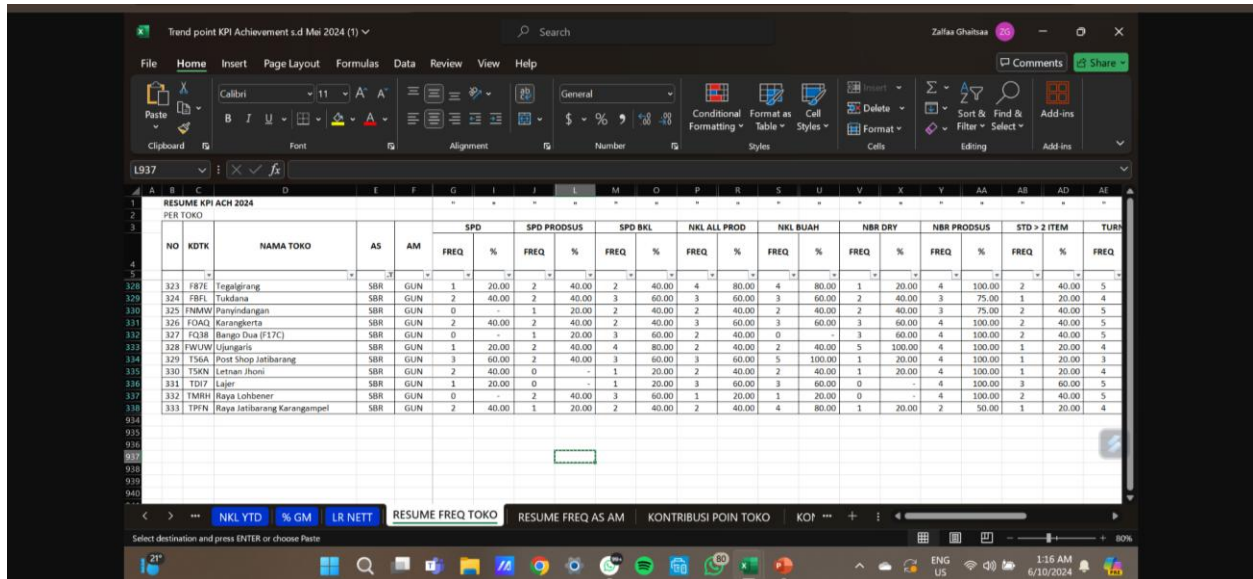
Data laporan manual

Tanggal	Keterangan	Jenis	Jumlah	Bukti Transaksi	Aksi
2026-04-02	Gula	Pengeluaran	12000	Lihat	Edit Hapus
2026-04-02	Terigu	Pengeluaran	15000	Lihat	Edit Hapus
2026-04-01	Beras	Pemasukan	60000	Lihat	Edit Hapus

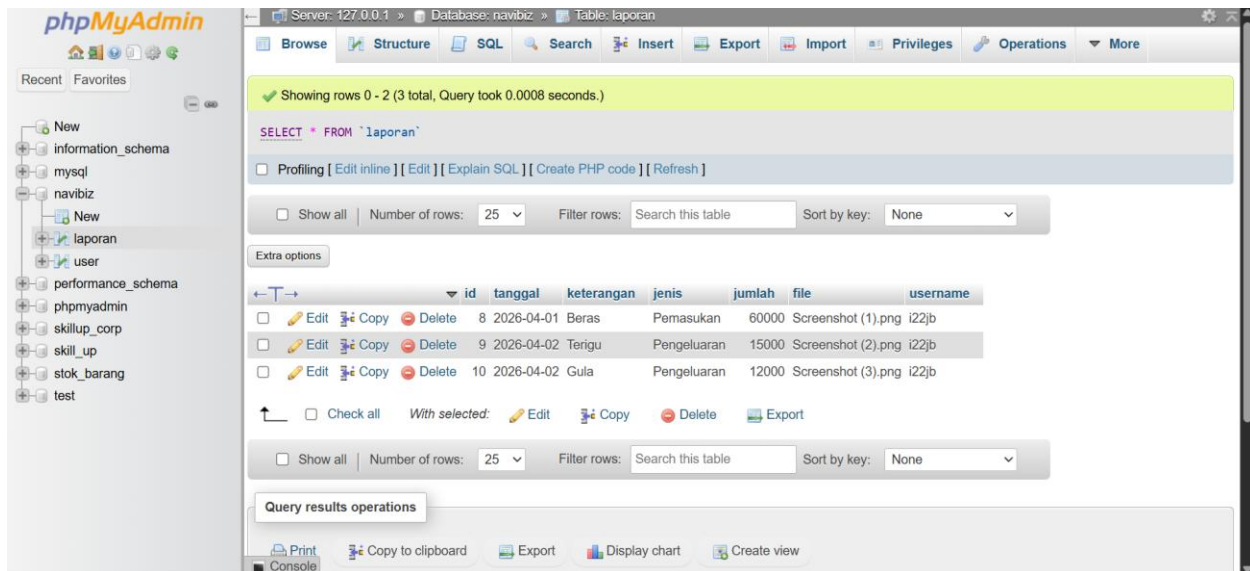
Gambar D.1.3 Tampilan saat sudah edit



Gambar D.1.4 Tampilan saat ingin upload file



Gambar D.1.5 Tampilan saat tulisan “Lihat” di pencet



Gambar D.1.6 Tabel setelah di edit

E. ANALISIS DAN PEMABAHASAN

Pada praktikum ini, dibuat sebuah aplikasi web sederhana berbasis PHP yang menerapkan konsep Pemrograman Berorientasi Objek (OOP). Sistem yang dibangun mencakup fitur login, registrasi, serta pengelolaan data laporan berupa Create, Read, Update, Delete (CRUD) dan upload file. Struktur program disusun menggunakan beberapa class seperti Database, Auth, Session, User, dan Laporan, yang masing-masing memiliki tanggung jawab berbeda. Class Database digunakan untuk mengelola koneksi ke database, sedangkan class lain menerima koneksi tersebut melalui constructor sehingga tidak perlu membuat koneksi berulang. Hal ini menunjukkan penerapan konsep OOP yang membuat kode lebih terstruktur dan mudah dikelola.

Dalam implementasinya, proses autentikasi dilakukan oleh class Auth yang mengatur login dan logout pengguna dengan memanfaatkan session dan cookie. Setelah login berhasil, data pengguna disimpan dalam session sehingga dapat digunakan untuk mengakses halaman lain seperti dashboard. Class Session berfungsi untuk memastikan bahwa hanya pengguna yang sudah login yang dapat mengakses halaman tertentu, sehingga keamanan akses dapat terjaga. Selain itu, fitur registrasi ditangani oleh class User, yang melakukan pengecekan terhadap username agar tidak terjadi duplikasi sebelum menyimpan data ke database.

Pengelolaan data laporan dilakukan oleh class Laporan yang mengimplementasikan fungsi CRUD secara lengkap. Data yang ditampilkan difilter berdasarkan username sehingga setiap pengguna hanya dapat melihat dan mengelola datanya sendiri. Fitur upload file juga telah terintegrasi dalam proses penambahan dan pembaruan data, di mana file yang diunggah akan disimpan ke dalam folder tertentu dan namanya dicatat di database. Hal ini

menunjukkan bahwa sistem tidak hanya mengelola data teks, tetapi juga file secara bersamaan dalam satu proses.

Secara keseluruhan, aplikasi yang dibuat sudah berhasil menerapkan konsep OOP dengan baik, ditandai dengan penggunaan class, object, constructor, dan enkapsulasi. Kode menjadi lebih modular, terorganisir, dan mudah dikembangkan. Namun, masih terdapat beberapa kekurangan seperti belum adanya enkripsi password, belum menggunakan prepared statement untuk mencegah SQL injection, serta validasi input yang masih terbatas. Meskipun demikian, sistem sudah dapat berjalan dengan baik sesuai dengan tujuan praktikum, yaitu membangun aplikasi web berbasis OOP dengan fitur utama yang lengkap.

F. KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pembuatan aplikasi web berbasis PHP dengan menerapkan konsep Pemrograman Berorientasi Objek (OOP) berhasil dilakukan dengan baik. Sistem yang dibangun telah memiliki fitur utama seperti registrasi, login, serta pengelolaan data laporan menggunakan operasi CRUD dan upload file. Penerapan OOP melalui penggunaan class, object, constructor, dan enkapsulasi membuat struktur kode menjadi lebih rapi, terorganisir, dan mudah untuk dikembangkan. Selain itu, penggunaan session dan cookie memungkinkan pengelolaan autentikasi pengguna berjalan dengan baik sehingga keamanan akses halaman dapat terjaga. Meskipun masih terdapat beberapa kekurangan seperti aspek keamanan yang belum optimal dan validasi input yang masih terbatas, secara keseluruhan aplikasi telah memenuhi tujuan praktikum dan menunjukkan pemahaman dasar yang baik dalam penerapan OOP pada pengembangan web.

G. LAMPIRAN

<https://github.com/AriqAhmadNayaka/WebPro4904.git>